



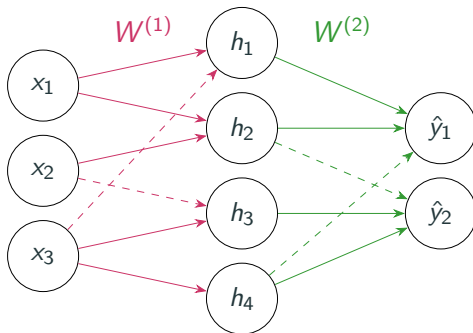
Spectral Higher-Order Neural Networks

Gianluca Peri¹ Timoteo Carletti² Duccio Fanelli¹ Diego Febbe¹

¹ University of Florence ² University of Namur

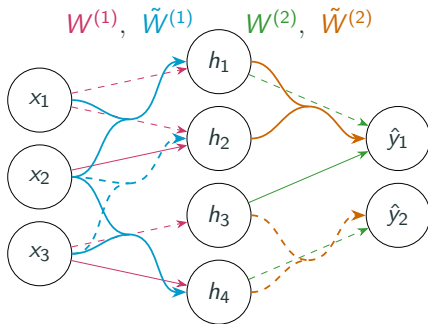


Standard Neural Networks (a.k.a. MLPs)



$$\hat{y}_l = \sum_k W_{lk}^{(2)} \underbrace{\sigma \left(\sum_i W_{ki}^{(1)} x_i \right)}_{h_k}, \quad O(N^2) \text{ scaling}$$

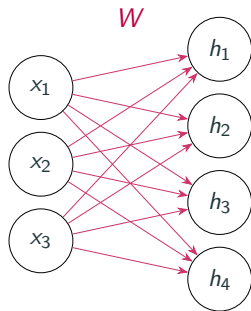
Higher-Order Neural Networks (HONNs)



$$h_k = \sigma \left(\sum_i W_{ki}^{(1)} x_i + \sum_{ij} \tilde{W}_{kij}^{(1)} x_i x_j \right), \quad \text{bad } O(N^3) \text{ scaling}$$
$$\hat{y}_l = \sum_k W_{lk}^{(2)} h_k + \sum_{km} \tilde{W}_{lkm}^{(2)} h_k h_m$$

The concept dates back to **Sigma-Pi Units**: D. E. Rumelhart, G. E. Hinton, J. L. McClelland, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, MIT Press, 1986.

Standard Neural Networks and Adjacency Matrices



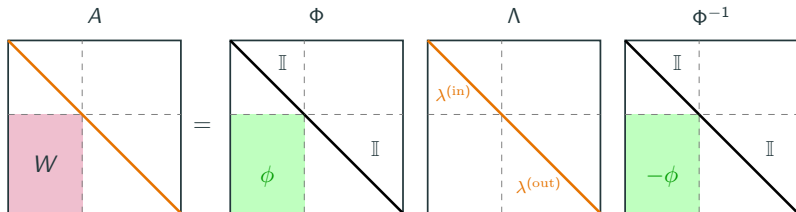
directed bipartite graph

$$W \times \mathbf{x} = \mathbf{h}$$

Matrix multiplication diagram for a standard neural network. Matrix A is partitioned into W (pink) and zeros. Vector $\mathbf{a}$ is partitioned into $\mathbf{x}$ (blue) and 0 (white). The result vector $A\mathbf{a}$ is partitioned into 0 (white) and $\mathbf{h}$ (purple).

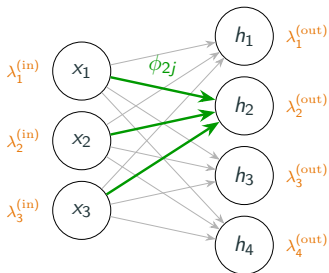
Matrix multiplication diagram for a neural network with a bias term. Matrix A has a diagonal line and is partitioned into W (pink) and zeros. Vector $\mathbf{a}$ is partitioned into $\mathbf{x}$ (blue) and 0 (white). The result vector $A\mathbf{a}$ is partitioned into $*$ (grey) and $\mathbf{h}$ (purple).

Spectral Parameterization for Standard Neural Networks



$$w_{ij} = (\lambda_j^{(\text{in})} - \lambda_i^{(\text{out})}) \phi_{ij}$$

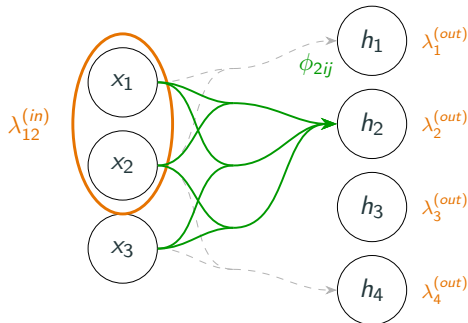
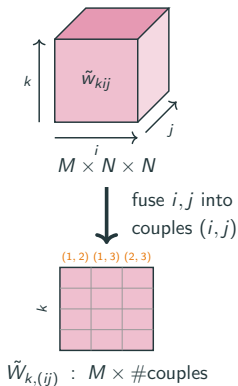
Train eigenvalues, eigenvectors, or both!



one **eigenvalue** per neuron

one **eigenvector** per bundle

Spectral Higher-Order Neural Networks (SHONNs)



one $\lambda^{(in)}$ per couple of inputs,
one $\lambda^{(out)}$ per output neuron

$$\tilde{W}_{kij} = (\lambda_{ij}^{(in)} - \lambda_k^{(out)}) \phi_{kij}$$

We can recover $O(N^2)$ by training only the eigenvalues!

SHONNs Can Augment Other Neural Models

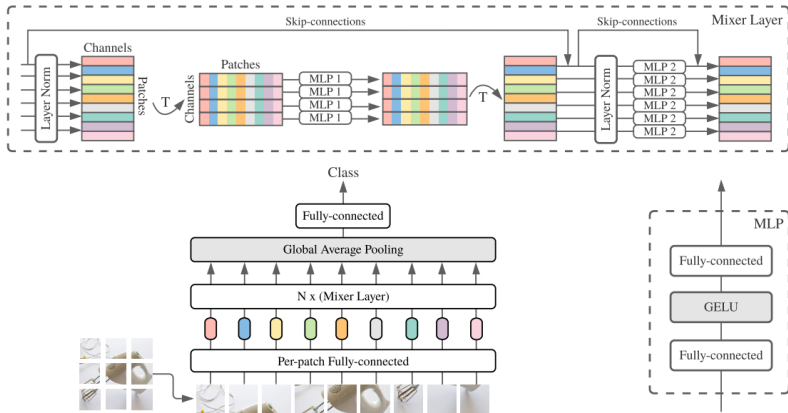


Figure 1: Depiction of a standard MLP-Mixer: (1.) The image is divided in patches (different colors in the schematics). (2.) The RGB patches are projected into an higher feature dimension (C channels) (3.) The Mixer Layers make MLPs operate first patch-wise (fixed channel) and then channel-wise (fixed patch) (4.) In the end we average over the patches to get a single vector of length C (5.) Finally we use a linear classifier to select the correct class for the image.

SHONNs Can Augment Other Neural Models

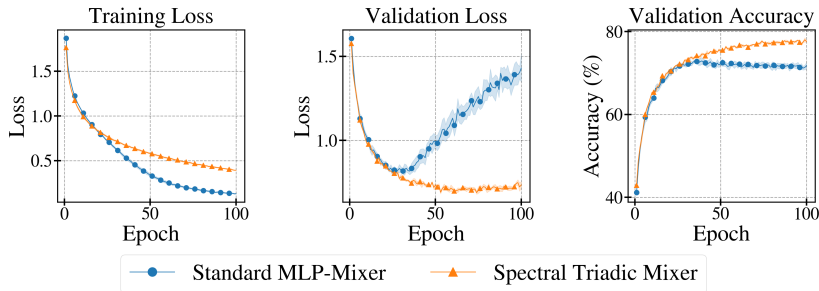


Figure 2: Spectral Triadic MLP-Mixer (each MLP gets swapped with a 2-layer SHONN), without triadic eigenvectors training and without activation functions (recovers $O(N^2)$ scaling) vs standard MLP-Mixer. Training only on CIFAR-10 (no pre-training). We can see the higher order elements acting as an implicit regularizer, appropriately preventing overfitting.

N-Bits Parity Problems

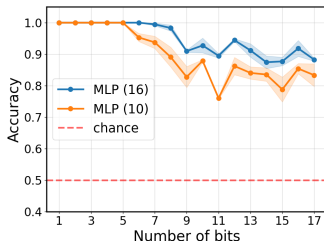
Definition: Given an input vector $\mathbf{s} = (s_1, \dots, s_N) \in \{+1, -1\}^N$, classify it as **odd** (class $y = 0$) or **even** (class $y = 1$):

$$y(\mathbf{s}) = \frac{1}{2} \left(\prod_{i=1}^N s_i + 1 \right)$$

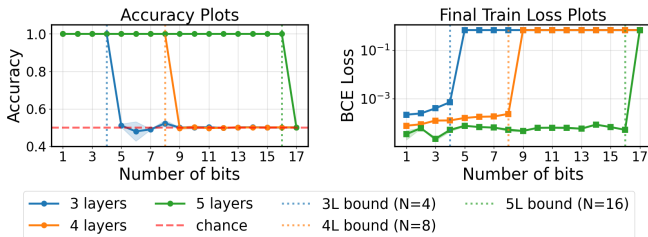
Why it matters in ML? It is a standard *hard* benchmark for neural networks:

- ▶ **Highly linearly unsolvable** even in the $N=2$ case, we get the *xor* problem, which no perceptron can solve.
- ▶ **Maximal sensitivity** i.e. flipping *any* bit flips the label, so the model must properly attend to *every* input feature.

SHONNs Theoretical Expressivity is easy to control



(a) MLPs on N -bits parity problems.



(b) SHONNs on the same parity problems.

Thank you!

gianluca.peri@unifi.it



Scan for preprint!